

CS 3502 Project 1: Multi-Threaded Programming

Lizbeth Ochoa

March 7, 2026

1 Phase 1 : Race Condition

1.1 Program Description

Phase 1 demonstrates race conditions using multiple teller threads in an accounting system. The teller threads perform deposits and withdrawals on a shared bank accounts with synchronization.

1.2 Program Outputs

1.3 Reflection Questions

Q1.1 - Race Condition Evidence

Run	Final total Balance
1	2130
2	1791
3	1924
4	1782
5	2069

Answer: After running the program 5 times, it is clear that there is a different total each time. The program produces a different total because multiple threads simultaneously access and modify the shared account balances. This is without synchronization. The race condition occurs when the threads read and update the same memory at the same time. The critical section occurs in these lines of code:

```
double current_balance = accounts[account_id].balance;  
accounts[account_id].balance = current_balance + amount;
```

Q1.2 Thread vs Process

Answer: A process is its own program with its own memory space and resources. While a thread is a smaller unit of execution within a process. Threads share the same memory space as other threads in the same process which is what connects to race conditions.

Q1.3 pthread_join Purpose

Answer: When I removed the pthread_join calls and ran my program, the main thread terminates and does not wait for the others. So the program becomes an incomplete output. The call is necessary because it makes sure that the main function waits for the rest of the threads to finish.

Q1.4 Non-Determinism

Answer: Non-deterministic behavior means that a program with the same inputs produces different results each time it is ran. This occurs in concurrent programs because the scheduler is what determines which thread executes first, which leads to different results. This makes debugging difficult because an error could not be easily noticed.

2 Phase 2 : Mutex Synchronization

2.1 Program Description

Phase 2 improved our program from Phase 1 by including mutex locks to protect shared data. Each bank account now has mutex locks which a thread must acquire before modifying the account balance. After acquiring the lock and making their changes to the account, the lock is released. This prevents race conditions and ensures only one thread at a time can change an account.

2.2 Reflection Questions

Q2.1 Correctness Verification

Answer: After running the program 10 times, the total balance remained the same each time. This shows how the race conditions from phase 1 was resolved. In Phase 1, we recieved different totals each time we ran the program which showed how each thread raced and caused the data to be inconsistent. In Phase 2, the mutex locks do not allow that and only allow one thread at a time. This ensured more consistent results from our program.

Q2.2 Critical Section

Answer: A critical section is the part of a code where shared resources are accessed or modified. If multiple threads enter this section at the same time, race conditions occur. In Phase 2 program, the critical sections occurred in side the functions of deposit safe and withdrawal safe. These sections include how the account balances were updated. The mutex locks were inserted before and after to make sure a lock is aquired and released before entering the critical section.

Q2.3 Performance Tradeoffs

Answer: Phase 2 program runs slower because of the mutex locks. Each thread must wait for a lock to be aquired after another thread has released the lock. This causes a small wait although is worth it. This is acceptable because it outputs correct results from the program which is needed more than maximum speed. The shared resources must be protected so mutiple threads do not alter it at the same time causing incorrect data.

Q2.4 Granularity

Answer: Each account has its own mutex to allow multiple threads to alter each account simultaneously. If a signle global mutex were used for all accounts, only one thread could access any account at a time. This would really slow down a program because all threads would have to wait even when they are wanting to alter a different account.